name: eshop-automation-23127259 description: Build, repair, and audit SRS-driven Playwright suites for the EShop SUT when tests must use POM, external CSV/JSON, intentional @bug failures, and independent Chromium/Firefox/WebKit reports. Do not use to modify the SUT merely to make tests pass.

EShop feature automation

Workflow

1. Read specification before code

Extract the acceptance criteria for the selected feature and relevant cross-cutting requirements (FR-21 to FR-24 and SEC rules). Expected results come from the SRS, never from the current faulty implementation.

2. Probe real DOM, routes, APIs, and state lifetime

Inspect the page and source to find stable selectors and understand SPA state. Never assume labels have `for`, email fields use `type=email`, or data survives `page.goto()`. Build a disposable probe when behavior is ambiguous.

3. Externalize every test value

- CSV: repeated matrices such as credentials, products, boundaries.
- JSON: messages, expected labels, API payloads, role/security cases.

Importing an unused file does not count as DDT. Inline data arrays/objects in a spec are not accepted.

4. Use Page Objects without hiding assertions

Page Objects own locators, navigation, and reusable actions. Specs own SRS assertions. Prefer web-first locators and SPA link clicks when React Context must survive.

5. Gate ordinary cases before accepting red cases

Run:

```
npx playwright test <spec> --project=chromium --grep-invert @bug
```

Every ordinary case must pass. Wrong selectors, browser-launch errors, lost state, and timeouts are automation defects, not SUT bugs.

6. Tag genuine defects

Use both a visible title marker and Playwright metadata:

```
test('TCxx: @bug SRS expectation', { tag: '@bug' }, async ({ page }) => {
  // Assert the SRS behavior; do not weaken it to match the SUT.
});
```

7. Run three engines and verify reports

```
node scripts/run-multibrowser.mjs <spec>
node scripts/verify-report-banner.mjs
```

Require Chromium, Firefox, and WebKit; one HTML report per engine; `Run by: 23127259`; ISO timestamp; and `unexpectedFailures: 0`.

8. Record defects and human review

For each root defect, record severity, SRS, test mapping, steps, expected, actual, screenshot/evidence, proposed fix, and GitHub Issue. Record what the AI got wrong and how human review corrected it.

EShop facts worth preserving

- Backend: `http://localhost:3000`; Web: `:5173`; Admin: `:5174`.
- Admin login DOM uses placeholders `Email` / `Password` and button text `Login`.
- Admin import controls appear only after selecting the Products tab.
- Cart state lives in React Context and is lost on full reload; use in-app routing after adding products.
- Scope FR-16 preview locators to the import panel because the Products page has multiple tables.
- `scripts/run-multibrowser.mjs` separates `bugFailures` from `unexpectedFailures`.

Deliverables checklist

- [ ] At least 12 cases for the feature.

- [ ] External CSV/JSON actually drives tests.
- [ ] At least three assertion families.
- [ ] Non-`@bug` gate is green.
- [ ] Three engines have consistent results.
- [ ] Three report banners show student ID and ISO time.
- [ ] Each defect has evidence and a GitHub Issue.
- [ ] Main report documents AI mistakes and human corrections.